

Sistemas Operativos

Práctica 1

Wenjie Chen - 100522255

Daniel Deblas Payo - 100522144

Xiya Ye - 100522159

3 de marzo del 2025

Índice

Descripción del Código.....	2
crear.c.....	2
combine.c.....	3
Batería de Pruebas.....	5
crear.c.....	5
Caso de prueba 1.....	5
Caso de prueba 2.....	5
Caso de prueba 3.....	5
Caso de prueba 4.....	6
Caso de prueba 5.....	6
combine.c.....	7
Caso de prueba 1.....	7
Caso de prueba 2.....	7
Caso de prueba 3.....	7
Caso de prueba 4.....	8
Caso de prueba 5.....	8
Caso de prueba 6.....	9
Caso de prueba 7.....	9
Corrector proporcionado.....	9
Conclusión.....	10

Descripción del Código

crear.c

El programa de *crear.c*, como su nombre indica, crea un fichero nuevo con los permisos que le indiquemos. Por lo que recibe `./crear <nombre_fichero> <modo>` para ejecutar, que serían 3 argumentos de entrada.

Lo primero que hay que añadir en el programa son las inclusiones de bibliotecas para poder usar posteriormente por ejemplo llamadas al sistema correctamente. Una vez tenemos esto, empieza la parte del *main* de nuestro código. Comenzamos comprobando que el número de argumentos de entrada es el correcto, en este caso verificamos que `argc` es igual a 3, dado que en caso de que no lo sea daría error y devuelve -1.

Antes de seguir, debemos comprobar que los permisos de entrada estén en formato octal, esto lo hacemos usando un bucle para recorrer la cadena. Entonces, empleamos la función *strchr* que nos dice la primera ocurrencia de un carácter, por lo que le pedimos la primera ocurrencia del carácter que estamos comprobando en la cadena "01234567", y si no lo encuentra entonces es incorrecto, así que devuelve -1.

Para asegurarnos de que los permisos que le vamos a asignar al fichero no se vean afectados por la máscara por defecto del sistema operativo, usamos *umask* y asignamos esta máscara a una variable de tipo *mode_t*, para más adelante, cuando hayamos terminado de crear el fichero, restaurar correctamente la máscara original.

Como los permisos pedidos, usamos *sscanf* para asignar el modo (`argv[2]`) en formato octal en la dirección de nuestra variable de tipo *mode_t*. Con esto, podemos crear el fichero pedido con la llamada al sistema *creat*, para eso le damos la ruta del archivo que queremos crear y el modo que hemos creado anteriormente. Para verificar que se ha creado bien, ponemos un condicional donde ponemos que si el resultado de *creat* es menor que 0 significa que no se ha creado bien, por lo que se imprime un mensaje de error (con *perror*) y devuelve -1.

Una vez creado el fichero, restauramos la máscara original con *umask*. Debido a que la función de *umask* da fallos en Guernika, tenemos que usar la función de *chmod* para que cambie los permisos a los que queremos.

Si todo funciona correctamente, significa que se ha creado el fichero con el modo pedido bien, por lo que devolvemos 0.

combine.c

El programa *combine.c* recibe 4 argumentos de entrada: *./combine <fichero 1> <fichero 2> <fichero salida>*, donde los ficheros 1 y 2 contienen información de las notas de cada alumno en un curso, y el programa se encarga de almacenar la combinación de ambos de manera ordenada en un tercer fichero (el fichero de salida). También crea un segundo fichero donde guarda las estadísticas de las notas de todos los alumnos.

Antes de todo, hemos añadido las inclusiones de biblioteca para posteriormente poder hacer llamadas al sistema, y también definimos el número límite de alumnos que puede leer, en este caso, 100 alumnos. Definimos la estructura de alumno, compuesta por diferentes campos, permitiéndonos organizar la información de los alumnos de manera eficiente.

Una vez empezamos la parte del main, lo primero que hacemos es asegurarnos de que haya recibido el número de argumentos correcto, esto lo hacemos con un condicional comparando *argc*. Posteriormente, definimos nuestras variables, 3 enteros para los 3 ficheros con los que trabajaremos primero, y un array con un tamaño donde puedan almacenar de máximo 100 alumnos (nuestro número límite de alumnos).

Lo siguiente que hacemos es empezar a leer el primer archivo, para ello usamos primero la llamada *open* para abrirlo y entonces usamos *read* para leer el fichero. Con la ayuda de un bucle, se lee el fichero de alumno en alumno y los va almacenando en un array. Mientras, debe ir asegurándose de que el número de alumnos leídos no superen el límite, por lo que implementamos una sentencia *if/else* para cubrir esto. Además, siempre que usamos un *if* para detectar errores, tenemos en cuenta que debemos cerrar los ficheros abiertos con *close*.

Una vez leído el primer fichero, hacemos lo mismo para leer el segundo fichero, teniendo en cuenta que el número de alumnos leídos del primer fichero.

Después de haber leído los dos ficheros y tener los alumnos almacenados en el array procedemos a ordenarlos. Para ello, en primer lugar, declaramos una variable auxiliar de tipo *struct alumno* en la que poder almacenar temporalmente a los alumnos. Posteriormente, para realizar la ordenación de los alumnos en la lista hemos implementado el algoritmo de la burbuja sobre el array. Con el primer bucle *for* recorremos la lista mientras que con el segundo, comparamos los alumnos consecutivos en la lista. Si el alumno *j* tiene una nota mayor que el alumno *j + 1* se intercambia la posición en el array asegurando así que se ordenan en el orden correcto. Después de que se haya aplicado el algoritmo sobre la lista de alumnos, se obtiene como resultado una lista con todos los alumnos iniciales ordenados de menor a mayor en función de su nota. Tras esto, pasamos a la creación del fichero *estadística.csv*.



Creamos un fichero llamado *estadística.csv* tal y como nos indica en el enunciado, en el cual si el archivo no existe se crea en modo escritura y en caso de que ya exista se vacía; permitiendo la lectura y escritura para nosotros y solo lectura para otros. Una vez creada, verificamos si hay algún error al abrirlo y en caso de que haya fallo finalizar el programa.

Creamos cinco variables iniciadas a valor cero y usamos un bucle para ir hacer conteo de las notas de cada tipo, cada una con su condición. Para imprimir por pantalla usamos `printf`, usamos `%d` en el cual se sustituye por el signo adecuado, `%.2f%%` para que salga la solución con 2 decimales y `\n` para un salto de línea. Luego, usamos *sprintf* para formatear una cadena de caracteres y *write* para escribir en el fichero *estadísticas.csv* y comprobamos para cada caso si se escriben correctamente. Por último, cerramos el fichero y retornamos un cero ya que al llegar hasta el final el programa es correcto.

Batería de Pruebas

crear.c

Caso de prueba 1

- Comando a ejecutar: `./crear prueba1 0644`
- Descripción de prueba: creación de un fichero con permisos válidos.
- Resultado:

```
-rw-r--r-- 1 a0522159 alumnos 0 mar 2 15:09 prueba1
```

Caso de prueba 2

- Comando a ejecutar:
 - `./crear`
 - `./crear prueba2`
- Descripción de prueba: creación de un fichero sin argumentos suficientes.
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacrear$ ./crear  
Error: número de argumentos incorrecto
```

```
a0522159@guernika-gpu:~/ssoo/pruebacrear$ ./crear prueba2  
Error: número de argumentos incorrecto
```

Caso de prueba 3

- Comando a ejecutar:
 - `./crear prueba3 hola`
 - `./crear prueba3 0789`
- Descripción de prueba: creación de un fichero con el argumentos de permisos inválidos, en vez de número en octal, con letras.
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacrear$ ./crear prueba3 hola  
Error: formato de permisos incorrecto, debe ser un número en octal  
a0522159@guernika-gpu:~/ssoo/pruebacrear$ ./crear prueba3 0789  
Error: formato de permisos incorrecto, debe ser un número en octal
```

Caso de prueba 4

- Comando a ejecutar: ./crear prueba4 0777
- Descripción de prueba: creación de ficheros con todos los permisos para todos
- Resultado:

```
-rwxrwxrwx 1 a0522159 alumnos 0 mar 2 15:50 prueba4
```

Caso de prueba 5

- Comando a ejecutar: ./crear prueba5 000
- Descripción de prueba: creación de ficheros sin permisos
- Resultado:

```
----- 1 a0522159 alumnos 0 mar 2 15:52 prueba5
```

combine.c

Caso de prueba 1

- Entrada:
 - ./combine
 - ./combine curso1
 - ./combine curso2
- Descripción de prueba: falta de argumentos
- Salida:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine
Error: número de argumentos incorrecto
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1
Error: número de argumentos incorrecto
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2
Error: número de argumentos incorrecto
```

Caso de prueba 2

- Entrada: ./combine curso1 curso2 cursoSal
- Descripción de prueba: combinar archivos inexistentes
- Salida:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
open : No such file or directory
```

Caso de prueba 3

- Entrada:

fichero 1

```
struct alumnos lista[] = {
    {"Juan Perez", 5, 1},
    {"Maria Lopez", 9, 2},
    {"Carlos Garcia", 7, 1},
    {"Ana Fernandez", 4, 3}
```

fichero 2

```
struct alumnos lista[] = {
    {"Carlos Mendoza", 6, 2},
    {"Andrés Ramírez", 10, 2},
    {"Lucía Fernández", 3, 1},
    {"Elena Soto", 4, 3},
    {"Javier Morales", 8, 1}
```

```
};
```

- Descripción de prueba: combinar ficheros con datos de alumnos válidos

- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
M;1;11.11%
S;1;11.11%
N;2;22.22%
A;2;22.22%
F;3;33.33%
a0522159@guernika-gpu:~/ssoo/pruebacombine$ cat cursoSal
Lucía FernándezAna FernandezElena SotoJuan PerezCarlos MendozaCarlos GarciaJavier MoraleMaria Lopez      Andrés Ramírez
```

Caso de prueba 4

- Entrada:
 - fichero 1: lista de 50 alumnos diferentes, todos con la misma nota (10).
 - fichero 2: lista diferente de 50 alumnos, todos con la misma nota (10).
- Descripción de prueba: combinar dos ficheros con el límite número de alumnos posibles a leer
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
M;100;100.00%
S;0;0.00%
N;0;0.00%
A;0;0.00%
F;0;0.00%
```

Y en el fichero de salida hay 100 alumnos.

Caso de prueba 5

- Entrada:
 - fichero 1: lista de 50 alumnos diferentes, todos con la misma nota (10).
 - fichero 2: lista diferente de 50 alumnos, todos con la misma nota (9).
- Descripción de prueba: combinar dos ficheros, donde hay 100 alumnos en total, con la mitad de alumnos con una nota y la otra con otra nota.
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
M;50;50.00%
S;50;50.00%
N;0;0.00%
A;0;0.00%
F;0;0.00%
```

Y en el fichero de salida hay 100 alumnos ordenados, primero los del fichero 2 y después los alumnos del fichero 1.

Caso de prueba 6

- Entrada:
 - fichero 1: lista de 50 alumnos diferentes, con notas aleatorias.
 - fichero 2: lista diferente de 51 alumnos, con notas aleatorias.
- Descripción de prueba: combinar dos ficheros, donde hay 101 alumnos en total.
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
Error: se ha superado el número max de alumnos
```

Caso de prueba 7

- Entrada:

fichero 1

fichero 2

```
struct alumnos lista[] = {
    {"Tomás Ibáñez", 10, 2},
    {"Alejandra Carrasco", 5, 1},
    {"Francisco Núñez", 0, 4},
    {"Eva Del Río", 7, 1},
    {"Martín Cabrera", 11, 1},
    {"Claudia Fuentes", 3, 2}
};
```

```
struct alumnos lista[] = {
    {"Antonio Marín", 1, 1},
    {"Rosa Vega", 9, 1},
    {"Federico Roldán", 7, 1},
    {"Angela Bautista", 2, 1},
    {"Emilio Rosales", 10, 1},
    {"Verónica Tapia", 9, 1}
};
```

- Descripción de prueba: combinar dos ficheros, donde un alumno tiene una nota 11 (inválido).
- Resultado:

```
a0522159@guernika-gpu:~/ssoo/pruebacombine$ ./combine curso1 curso2 cursoSal
Error: nota inválida
```

*Todos los casos se han probado en Guernika

Corrector proporcionado

Además de pasar los casos de prueba que hemos propuesto, ambos programas pasan también los tests que se nos han proporcionado.

```
xiya@xiya-VirtualBox:~/ssoo/P1/probador$ python3 probador-ssoo-p1.py ssao_p1_100522159_100522144_100522255.zip
CORRECTOR: corrigiendo ssao_p1_100522159_100522144_100522255.zip
CORRECTOR: NIA 1 100522159 NIA 2 100522144 NIA 3 100522255
CORRECTOR: descomprimiendo
gcc -Wall -Werror -Wextra -O2 -c crear.c -o crear.o
gcc -Wall -Werror -Wextra -O2 -o crear crear.o
gcc -Wall -Werror -Wextra -O2 -c combine.c -o combine.o
gcc -Wall -Werror -Wextra -O2 -o combine combine.o
Compilación correcta
Test 1: OK
Test 2: OK
Test 3: OK
Test 4: OK
Test 5-a: OK
Test 5-b: OK
Test 6-a: OK
Test 6-b: OK
1 1 1 1 1 1 1
```

Conclusión

La realización de este trabajo ha sido de gran ayuda para entender cuál es el funcionamiento de los ficheros y el funcionamiento de los mismos con el sistema operativo orientado a un ejemplo real. En primer lugar, hemos podido comprender cómo se utilizan y cual es la función de llamadas al sistema. Por otro lado, gracias a la creación del programa *crear.c* hemos podido afianzar nuestros conocimientos sobre los permisos en los ficheros. Para así, poder hacer el programa *combine.c* que es más complejo, necesitando usar diferentes llamadas al sistema.

Algunos problemas que hemos encontrado a la hora de implementar el código de *crear.c* ha sido entender la máscara del sistema, ya que no sabíamos a qué se debía que no se nos crearan bien los archivos. Con la ayuda de la función *umask* hemos podido solucionarlo. Otro problema que hemos tenido con el modo del fichero creado, ha sido que no nos funcionaba en Guernika, ya que esta función que hemos implementado no funciona correctamente en Guernika, por lo que hemos tenido que usar *chmod* para arreglarlo, asignando directamente los permisos al fichero antes de finalizar el programa.

A la hora de implementar *combine.c*, hemos tenido problemas con la llamada al sistema *read*, ya que no sabíamos muy bien cómo funcionaba el buffer, pero ver algunos ejemplos y con la ayuda del manual lo hemos podido entender y podido implementarlo en nuestro código de manera eficiente. Otra duda que tuvimos fue darnos cuenta de que existe una función que nos permite introducir datos formateados en una cadena, por lo que lo hemos usado para hacer las estadísticas. Como necesitábamos una cadena de caracteres, pero nuestros datos eran de tipo entero, por lo que no podíamos imprimirlo como una cadena con los datos separados por “;”, así que usamos esta función para convertirlo en una cadena de caracteres.

Por último, gracias a la realización de la batería de pruebas hemos podido trabajar con el entorno y solucionar errores que se estaban cometiendo en la realización de los programas.